PERTEMUAN 12 · Praktikum 3

Parser Generator Flex/Bison dan Aksi Semantik

Universitas Bale Bandung

Fakultas Teknologi Informasi · Program Studi Teknik Informatika

Mata Kuliah Teknik Kompilasi (MWCA017) · Semester Ganjil · 3 SKS

Dosen: Cecep Suwanda, S.Si., M.Kom.

Capaian Pertemuan

CPMK induk: CPMK 04 — Mengimplementasikan parser sederhana untuk validasi kode.

Sub-CPMK yang dinilai

- **Sub-CPMK 4.3:** Mengimplementasikan parser subset bahasa C dengan Flex dan Bison beserta aksi semantik, dengan berkas kode dan keluaran program sebagai bukti.
- **Sub-CPMK 4.4:** Membuat AST dan validator kode untuk subset bahasa C, lalu menguji keluaran pesan kesalahan sintaksis yang informatif.

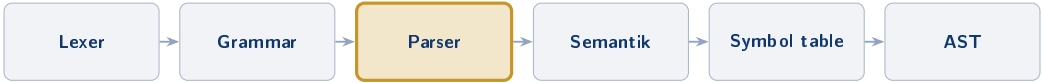
Praktikum 3 menilai 4.2, 4.3, dan 4.4; artefak 4.2 dari Pertemuan 11 ikut dikumpulkan.

Agenda Pertemuan (150 menit)

Menit	Kegiatan
20	Briefing Praktikum 3 dan cakupan artefak
25	Ceramah: arsitektur parser generator dan Bison
35	Praktik: deklarasi, aturan, dan aksi semantik
35	Praktik: integrasi Flex–Bison dan AST
25	Pengujian, diagnostik, dan perbandingan
10	Laporan dan penutup

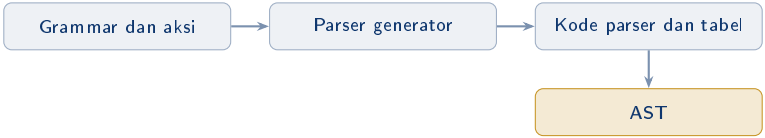
Porsi praktik 95 menit. Setiap mahasiswa menyerahkan grammar Bison, AST, dan keluaran pengujian.

Posisi Pertemuan pada Rantai Proyek



- Pertemuan ini menyelesaikan simpul **parser** dengan alat produksi: Bison membangun tabel LALR(1) dan AST.
- AST menjadi masukan analisis semantik pada Pertemuan 13–15.
- Praktikum 3 menghasilkan Milestone M3.

Arsitektur Parser Generator



- Pengguna menulis aturan produksi dan aksi semantik.
- Generator membangun tabel dan fungsi parser.
- Deskripsi bahasa terpisah dari algoritma parsing.
- Alat buku ini: Bison untuk parser, Flex untuk lexer.

Keuntungan dan Keterbatasan

Keuntungan

- mengurangi kode yang ditulis tangan;
- tabel dibangun sistematis;
- perubahan grammar mudah diterapkan;
- mendukung grammar lebih besar.

Keterbatasan

- bergantung pada alat;
- pesan kesalahan kadang kurang kontekstual;
- perlu belajar membaca laporan konflik;
- versi alat memengaruhi hasil.

Bison/Yacc memakai grammar LALR(1); Flex menghasilkan *yylex* yang dipanggil Bison.

Struktur Berkas Bison

1. **Deklarasi**: token, tipe, precedence, dan kode C.
2. **Aturan**: grammar dan *semantic action*.
3. **Kode pengguna**: fungsi bantu dan `yyerror`.

Bagian deklarasi

`%union` mendefinisikan tipe nilai semantik; `%token` mendeklarasikan token; `%type` menghubungkan nonterminal dengan tipe; `%left` dan `%right` menetapkan precedence serta associativity.

Deklarasi `parser.y`

```
%union {
    Token token;
    AstNode *node;
}

%token <token> T_INT T_FLOAT T_CHAR T_VOID
%token <token> T_IF T_ELSE T_WHILE T_FOR T_RETURN
    T_BREAK T_CONTINUE T_PRINT
%token <token> T_IDENT T_INT_LIT T_FLOAT_LIT
    T_CHAR_LIT T_STRING_LIT
%token <token> T_PLUS T_MINUS T_STAR T_SLASH
    T_PERCENT
%token <token> T_LT T_LE T_GT T_GE T_EQ T_NE T_AND
    T_OR T_NOT T_ASSIGN
%token <token> T_LPAREN T_RPAREN T_LBRACE T_RBRACE
    T_SEMI T_COMMA
%token <token> T_EOF

%type <node> program stmt_list stmt block
    assign_or_call
```

Listing 1: Deklarasi %union, token, dan %type

- Token membawa nilai bertipe `Token`.
- Nonterminal membawa `AstNode *`.
- `%type` menghubungkan keduanya.
- `%left` menetapkan precedence.

Aturan dan Aksi: program

```
53 program
54     : stmt_list T_EOF
55         {
56             $$ = ast_new(AST_PROGRAM, $2.token);
57             AstNode *child = $1;
58             while (child) {
59                 AstNode *next = child->next;
60                 child->next = NULL;
61                 ast_add_child($$, child);
62                 child = next;
63             }
64         }
65     ;
```

Listing 2: Aturan program dan aksinya

- Aksi membuat simpul AST_PROGRAM.
- Daftar sementara memakai next.
- Setiap anak dipindahkan ke children.
- $$ adalah hasil; $1, $2 adalah nilai simbol.

Nilai Semantik dan Pembangunan AST

- `$$`: nilai hasil nonterminal kiri.
- `$1`, `$2`, ...: nilai simbol ruas kanan.
- `$<token>1`: akses anggota union tertentu.
- Aksi dijalankan saat aturan direduksi.

Pola pembangunan AST

Buat simpul dengan `ast_new`, lalu tambahkan anak dengan `ast_add_child`. Daftar `next` dipakai selama parsing; `children` menyimpan struktur pohon final.

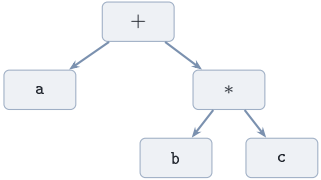
Aksi untuk Assignment, Call, dan Block

```
135 assign_or_call
136     : T_ASSIGN expr
137         {
138             $$ = ast_new(AST_ASSIGN, $1.token);
139             ast_add_child($$, $2);
140         }
141     | T_LPAREN args T_RPAREN
142         {
143             $$ = ast_new(AST_CALL, $1.token);
144             AstNode *arg = $2;
145             while (arg) {
146                 AstNode *next = arg->next;
147                 arg->next = NULL;
148                 ast_add_child($$, arg);
149                 arg = next;
```

Listing 3: Aksi `assign_or_call`

- `= expr` membangun simpul `AST_ASSIGN`.
- `( args )` membangun simpul `AST_CALL`.
- Argumen dipindahkan dari `next` ke `children`.
- Pola sama dipakai untuk blok.

AST untuk `a + b * c`



- Aksi `term` membangun $*$ untuk $b * c$.
- Aksi `additive` membangun $+$ dengan anak `a` dan simpul $*$.
- AST mencerminkan precedence tanpa tanda kurung.
- Simpul menyimpan token dan lokasi untuk diagnostik.

Lokasi dan Pesan Kesalahan

```
void yyerror(const char *msg) {
    fprintf(stderr, "%d:%d: %s\n",
            yylloc.first_line,
            yylloc.first_column, msg);
}
```

Listing 4: Fungsi `yyerror` dengan lokasi

- `%locations` menyediakan `@1`, `@2`, dan `@$`.
- Lokasi disalin ke simpul AST.
- Pesan memuat lokasi, penyebab, dan token ditemukan.
- "syntax error" saja tidak memenuhi Sub-CPMK 4.4.

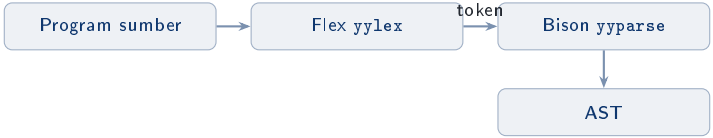
Komponen Pesan Kesalahan Sintaksis

Komponen	Contoh
Lokasi	baris 7, kolom 5
Penyebab	diharapkan ; setelah ekspresi
Token ditemukan	}
Saran perbaikan	tambahkan ; sebelum }

Pengujian

Pesan diuji pada kasus valid dan tidak valid. Diagnostik yang baik membantu pengguna menemukan kesalahan tanpa menebak.

Integrasi Flex–Bison



- `yylex` mengisi `yylval` dan mengembalikan kode token Bison.
- Driver memetakan jenis token lexer ke kode Bison.
- `yyparse` memakai tabel ACTION/GOTO LALR(1).

Langkah Build

Perintah build

```
flex -o build/lexer_flex.c src/lexer_flex.l
bison -d -o build/parser.tab.c src/parser.y
gcc -Iinclude -Ibuild build/lexer_flex.c build/parser.tab.c \
        src/token.c src/error.c src/ast.c src/main_bison.c -o build/scc
```

- Flex menghasilkan lexer C.
- Bison menghasilkan parser dan header token.
- GCC menautkan semuanya menjadi executable scc.
- Catat versi Flex, Bison, dan GCC pada laporan.

Driver Parser

```
int main(int argc, char **argv) {
    if (argc != 2) {
        fprintf(stderr, "penggunaan: %s berkas.sc\
            n",
                argv[0]);
        return 2;
    }
    if (!lexer_open(argv[1])) {
        return 2;
    }
    int status = yyparse();
    lexer_close();
    return status;
}
```

Listing 5: Driver yang memanggil yyparse

- Driver adalah alat uji.
- Membuka berkas dan menyiapkan lexer global.
- Memanggil yyparse.
- Mengembalikan status parsing.
- Pada milestone berikutnya, driver diganti alur semantik.

Bison dibandingkan Parser Tangan

Aspek	Bison	Parser tangan
Algoritma	LALR(1)	Recursive descent
Bentuk sumber	Berkas `.y`	Kode C langsung
Aksi semantik	`$$`, `$1`	Parameter dan nilai kembali
Konflik	Dilaporkan alat	Dikelola manual
Diagnostik	Perlu kode tambahan	Penuh, ditulis manual
Integrasi lexer	Flex	Pemanggilan fungsi manual
Cocok untuk	Grammar besar, proyek tim	Bahasa kecil, pembelajaran

Keduanya dapat dipakai bersama: Flex menangani token, Bison menangani struktur.

Pengujian Praktikum 3

Kasus valid

- deklarasi dan ekspresi;
- if/else dan while;
- panggilan fungsi;
- ekspresi campuran dengan kurung.

Kasus tidak valid

- titik koma hilang;
- kurung tidak seimbang;
- ekspresi tidak lengkap;
- deklarasi tanpa nama.

Minimal 20 kasus uji. Untuk setiap kasus tidak valid, periksa lokasi dan isi pesan.

Kegiatan Praktikum: M3

Langkah kerja

1. Lengkapi `src/parser.y`.
2. Definisikan `%union`, `%token`, dan `%type`.
3. Tulis *semantic action* untuk setiap aturan.
4. Integrasikan `yylex` dari lexer.
5. Jalankan Bison dan perbaiki konflik.
6. Uji dan bandingkan dengan parser tangan.

Kumpulkan tabel ACTION/GOTO SLR/LALR (4.2) dari Pertemuan 11 dan AST (4.4) yang dihasilkan hari ini.

Rubrik Praktikum 3

Aspek	Indikator	Bobot
Kebenaran	Grammar dan AST benar; konflik terselesaikan	40%
Kualitas kode	Aksi ringkas, penamaan, komentar, tanpa peringatan	25%
Kelengkapan	Ekspresi, pernyataan, deklarasi, dan AST	20%
Pengujian	Kasus normal, batas, dan kesalahan terdokumentasi	15%

Rubrik ini sama dengan rubrik artefak praktikum pada bagian 10 RPS.

Tugas dan Artefak

Artefak Sub-CPMK 4.3 dan 4.4

Parser Flex/Bison dengan aksi semantik, berkas kode, keluaran program, AST, dan pesan kesalahan sintaksis yang informatif.

- Grammar Bison tanpa konflik tak terduga.
- AST terbentuk untuk program valid.
- Minimal 20 kasus uji valid dan tidak valid.
- Diagnostik menyebut lokasi dan token yang diharapkan.
- Tabel ACTION/GOTO (4.2) dari Pertemuan 11 ikut dikumpulkan.
- Perbandingan dengan parser tangan terdokumentasi.
- Laporan singkat dan versi alat diserahkan.

Praktikum 3 berbobot 12% dan dilaporkan setelah Milestone M3 selesai.

Rangkuman dan Jembatan

- Parser generator memisahkan deskripsi grammar dari algoritma parsing.
- Bison memakai deklarasi token, tipe, precedence, aturan, dan aksi semantik.
- `$$` dan `$1` menghubungkan nilai simbol untuk membangun AST.
- Flex menyediakan `yylex`; Bison menyediakan `yyparse`; keduanya ditautkan menjadi satu executable.
- Pesan kesalahan harus menyebut lokasi, penyebab, dan token yang ditemukan.

Pertemuan berikutnya

Pertemuan 13 beralih ke analisis semantik: attribute grammar, *syntax-directed definition*, dan *translation scheme*. AST dari Bison menjadi masukan aturan semantik.